

Lecture 3

Recap & Notes:

Why is testing a sample?: testing is a sample; if we are going to test every case we need tons of unrealistic resources including time. So we choose some cases (extreme, overflow, false, logically wrong) cases to test against these chosen tests represents only a sample of the entire cases.

There are no (comprehensive) systems that are error-free! Nor can any test prove that they are error-free!: this means that no matter how good the software is tested we can never say that its error free. And note that: tests can show the presence of a bug not the absence of them.

What is the difference between a defect and a failure:

Defect: is a logical problem that is written in the code or in other project artifact, its still not executed yet and the user or the system that interacts with it cannot see it yet.

Failure: is the defect at execution, for instance; the program is being executed and returns an error or an unexpected output. The user, stakeholder, or other system can notice this problem.

What is the difference between Verification and Validation:

Verification: We check if what we have done is aligned with our requirements, we check if we have done it right. It deals with documentation and following the process that we have chosen.

Validation: We check if the feature or the function that we have developed is aligned with the requirements and does its intended function. Is it working right as it should be. It deals with the actual function and if it works as intended.

Ensuring quality costs money, but not as much money as releasing a low quality product.

The testing effort must be reasonably proportional to the achievable result.

Learning from Errors:

When an error occurs we can improve the process for 3 parts.

How can we prevent the error from happening again: this part where the constructive QA (build quality into the product by following software engineering and design practices) happens with organizational QA (improve the entire team and organization, maybe provide them some tools).

How can we catch the error: Analytical QA.

Debugging vs. Test:

Debugging: The process of locating, analyzing, and fixing defects in software once a failure has been observed. Its done manually by the developer, the developer finds the logical issue (locate it), then fix it to prevent it from causing a failure. Requirement met?

Testing: is the process of finding defects (static testing) or failures (dynamic testing) it's done by an automated test, or a QA tester. It only involves finding that there is an issue, no location or fixing. Usable?

Testing & Quality:

How does testing and quality relate to each other:

Testing measures quality the more testing issues you have the lower your product quality.

Testing increases the quality, when you find an error you fix it, document it, and add it the constructive process which over all increase your product quality and future product quality.

When testing the less errors you have the better the quality.

Analytical QA:

We can split the analytical QA into 2 parts

Process (Audit): This part of the analytical QA deals with the analysis part of organizational and constructive QA. We check if the roles and agreed documentation that we decided on in Organizational QA was followed. We also check if software design practices and architecture that we have agreed on in the Constructive QA was followed. We check if peer reviews were done for each module (part of constructive).

Results: This is where we actually do the reviews, the static analysis and the dynamic testing.

Testing Principles (Overview):

- 1) Testing shows the presence, not the absence of defects. (explained before)
- 2) Exhaustive testing is impossible. Testing everything is not feasible except in trivial cases. (explained before)
- 3) Early testing saves time and money. (explained before)
- 4) Defects cluster together: most of the product problems or errors comes from the same are for instance, 80% of your errors may come only from the payment module.
- 5) Tests wear out: running the same tests over and over again becomes less efficient by time.
- 6) Testing is context dependent: not all softwares need the same depth and the same testing some need less some need more. As an example; an airplane autopilot may need 10 times more formal in depth testing than a simple function in a web app.
- 7) Absence-of-defects fallacy: A defect free program is not necessarily usable. We may have an error free clock in-out app but it does not count for over time, then its not a quality product.

Expanding the testing principles:

- 1) **Testing shows the presence, not the absence of defects:** Testing reduces the amount of defects remaining undiscovered. That does not mean that tests can tell us the program is defect free.
- 2) **Exhaustive testing is impossible. Testing everything is not feasible except in trivial cases:** only in simple software you can test exhaustively, but in normal cases you

should aim for testing risky cases, edge cases, cases that might cause logical errors, test smart not hard.

- 3) **Early testing saves time and money:** When a defect is found earlier in the SDLC it means that it will cause much less trouble in future before we build a system on top of it. And by time the time and money spent on software quality will decrease, since we have many errors exposed in previous products. Thus, we shall start dynamic and static testing asap.
- 4) **Defects cluster together:** As explained before most of the errors come from small amount of modules, Therefore its very important to target modules that you believe might have most of the defects first.
- 5) **Tests wear out:** using the same test again and again will be less effective by time, you should change the test input or use another test. In most cases repeating the same test over and over again is very useless.
- 6) **Testing is context dependent:** There is no single universally applicable approach to testing. Testing is done differently in different contexts.
- 7) **Absence-of-defects fallacy:** having a system that is defect free does not guarentee that its of a high quality. We might build a system that is error free but does not fulfill the users' needs. So we fall behind, thus we shall also validate the system.

When do you stop testing?:

It's very important to know that when to stop testing is a decision call not a criteria and it depends on context but we overall have 4 rules to when to stop.

- All planned test cases in the analytical QA are done
- Test exit criteria are all met
- No more defects of high priority are being found
- The time, fund limit is reached.

Testing Efforts:

“Testing makes economic sense as long as the costs of finding and eliminating an error in the test are lower than the costs associated with the occurrence of an error during use.”: Testing the product or a module only makes sense if it costs less to fix it than to fix it after the release, and the opposite some fixes can wait since fixing it after deploying will cost less than delaying the deployment.

A good testing is like a liability insurance it costs real money but lets all stakeholders sleep tight.

You can't test everything because resources like time, money, and staff are limited. Waiting until the end of development makes this even harder. Start testing as soon as you can

Choose the type of test that matches the software and its goals. Not all tests are appropriate for every situation.

Don't waste time running tests that won't give new insights or detect defects.

Test cases should be updated as the software evolves to remain relevant and effective. Lets say you add a new feature in a module, the test cases should then be reviewed and updated to match the new feature for testing.

Consider both positive and negative test cases. Don't only test to validate the feature, or to check if a wrong input is noticed. You should make sure that if the system gets an unexpected input to fail safely. A wrong input should not be accepted but it should not also fail the system or cause data breach.

Test for unexpected behavior, edge cases, and potential security issues, even if they weren't specified in requirements.

As the software or the product evolves the stakeholders add more requirements both functional and non-functional, we must add test cases for those new requirements. And of course test them for quality. So the key idea here is to always reprioritize the tests when new requirements are added.

Criteria for Prioritizing:

- Focus on tests that if they fail would cause most harm. For example, a bank transaction module is of a higher testing priority than the feature that changes the background color to dark. Since failure of it causes severe harm.
- Focus on areas where probability of finding a failure will exist. For instance, a new developed feature by a junior using a new tech has higher chances of finding defects than a well company used technology feature developed by a senior.
- You should use a combination of both of the upper methods $\text{risk} = \text{damage} \times \text{probability}$.
- Even a minor defect can become a "show stopper" if it impacts trust, usability, or brand image. Even if its technically not severe it might have some other issues.
- If its not a key feature of the app it can wait and it has lower testing priority.
- Prioritize features that the costumer wants the most, meaning if a feature is main feature of your software like a check-out module on an e-commerce website it should has high priority.

- When you focus on main features regarding the non-func reqs. So if the performance of a functional feature is heavily needed to perform quickly and under stress you should test it using stress testing.
- From a development perspective you should test modules that affects other modules, central modules. Since fixing the defects their automatically fixes defects many other modules.
- Some parts of the project carry higher risks, you should test them first. Like a third party data input from google.
- Bugs are social, meaning like bacteria if you find a bug in a module there is a high chance you have other bugs in the same module or a similar one.
- Every time you add a requirement or discover a defect you should try to re-prioritize your test cases.

“Prioritize tests so that whenever you stop testing you have done the best testing in the time available.” This means prioritizing tests gives you some sort of a safe space, so if for some reason you have to stop testing. You’ve had fixed the most crucial defects and failures.

Terms in the ISTQB glossary:

1. **Test Basis:** The documentation or source used to derive what needs to be tested. Using a requirements document to design login tests.
2. **Test Execution:** The process of running test cases against the software and recording results. Manually entering invalid inputs into a form to see if validation works and recording the software reaction
3. **Schedule:** Planned timeline for performing testing activities. Performance tests scheduled for the last week of sprint 3.
4. **Test Condition:** Specific aspect, feature, or requirement to be verified, it means what you actually want to test. Example, Verify that the password field accepts a minimum of 8 characters.
5. **Test Case:** Detailed instructions to test a single scenario, including inputs, expected results, and execution steps. Example, Input username user1, password pass123, click login → expect “Welcome user1.”
6. **Test Run:** Executing a test case or a set of test cases once. Each execution counts as a separate run, results recorded for evaluation. Running the login test case today counts as one run.
7. **Test Suite:** Collection of related test cases grouped together for execution. This helps organize tests by feature, type, or priority. Example, All login-related tests form a login test suite.

8. **Test Script:** Detailed set of instructions or automated code to execute a test case or suite. Example, Automated script: Selenium or Cypress script that performs 50 login test cases automatically.
9. **Actual Result:** What really happens when a test case is executed compared against the expected result to determine pass/fail. Example, Expected login success, but actual result: "Error: invalid password."

Explaining Stupid Flowchart:

- 1) Test Basis : it's the starting point, it includes the requirements the user stories that we can later derive test cases from. Everything in testing is mainly derived from test basis (documentation of what we need, features, requirements and so on).
- 2) Test Condition: is an aspect (feature) from the test basis that need to be tested, a certain base (requirement) can derive many testing conditions.
- 3) Test Case: a test case a test derived from the test condition that is detailed including the expected result and what input should go in there. Includes also the steps.
- 4) Test Script: it's the executable code for the test, that specifies exact actions that we should perform (steps).
- 5) Test Suite: this is collection of test scripts that are categorized logically and are going to be executed together.
- 6) Test Execution Schedule: this plans when and what test suites are going to be executed.
- 7) Test Run: the actual test is being ran from the suite.
- 8) Actual result: the observed system behavior

2 Examples:

1) Test Basis

- Example 1: The functional requirement "The system must allow users to log in with a valid email and password."
- Example 2: A user story: "As a shopper, I want to add items to a shopping cart so that I can purchase them later."

2) Test Condition

- Example 1: Check that the login form accepts a valid email and password.
- Example 2: Verify that items can be successfully added to the shopping cart.

3) Test Case

- Example 1: Test Case for login:

- Steps: Enter valid email and password → click “Login”
 - Expected Result: User is redirected to the dashboard
- Example 2: Test Case for shopping cart:
 - Steps: Select an item → click “Add to Cart”
 - Expected Result: Item appears in the cart with correct quantity and price

4) Test Script

A test script is basically a detailed instruction manual for testing. It tells exactly what to do, in what order, and what to check.

- Example 1: Selenium script that inputs a username/password and clicks login.
- Example 2: Automated script that selects an item from a product list and adds it to the cart.

5) Test Suite

- Example 1: Login test suite containing multiple login test scripts (valid login, invalid login, password reset).
- Example 2: Shopping cart test suite including add item, remove item, update quantity, and view cart scripts.

6) Test Execution Schedule

- Example 1: Schedule to run login test suite every night at 2 AM.
- Example 2: Schedule to run the shopping cart test suite after each new build deployment.

7) Test Run

- Example 1: Executing the login test suite today at 2 PM on the staging environment.
- Example 2: Running the shopping cart test suite on the latest release candidate before production.

8) Actual Result

- Example 1: Observed that login succeeds and the dashboard loads correctly.
- Example 2: Observed that the item is added to the cart, but the quantity is incorrect.

N.B: Test basis & Test condition are part of test analysis. Test case is test design. test suite, test execution scheduler, test script are parts of test implementation. Test run and actual result are parts of execution.

Testing Activities CTFL:

In Order:

- Test planning
- Test monitoring and controlling
- Test analysis
- Test design
- Test implementation
- Test execution
- Test completion

Test Planning:

Activities:

Test Strategy Definition:

Defines testing objective, what do you want to achieve by testing. To make sure that certain functionality works. Define scope what will be tested and what not. Test core modules of a the bank transaction system. Identify risk of testing by categorizing the severity. Finally, specify test procedure; which teams are involved, what techniques will be used.

Test planning steps (more detailed):

Test resources (e.g. people, test environment, PCs): Decide on who and what is needed for the test. Example: team members, PCs, paid tools.

Which part of the system should be tested (with what intensity)?: for example the transaction module should be tested heavily while the UI module should be tested in low intensity

Which test procedures should be used? : Specify techniques for scanning, methods, and perhaps tool. As an example: Use sonarqube for static code scanning

Determine end criteria: Decide when testing is finished. For example, 80% of high criticality defects are done.

Prioritization of tests (accounting for the risk in the event of errors): prioritize tests based on the risk of the failure it carries

Plan tool support (deployment, procurement, training): decide on the tools that will be used for both testing, and the collaboration.

Determine rough test planning: create a timeline and order for what will be tested and when and for how long.

Results:

The test planning activities has many results, some of these results are called planning concepts, these shows some of the results of the planning activity. It summarize all decisions and arrangements.

Concepts are:

Background / Test Objectives

- Meaning: Explain why the testing is done and what it aims to achieve.
- Example: "Ensure that the payment module correctly handles all valid and invalid transactions."

Referenced Documents

- Meaning: List all sources used to design the tests.
- Example: Requirements specification, design documents, user stories, risk analysis.

Test Procedure

- Meaning: Describe how testing will be carried out.
- Example: Use black-box tests for user inputs, white-box tests for algorithms, stress testing for login module.

Reporting

- Meaning: Define how test results and defects will be documented and communicated.

Dependencies

- Meaning: Specify conditions or components needed before testing can proceed.

End Criteria

- Meaning: Define when testing is considered complete.

Test Monitoring and Control – Activities:

Test Monitoring: includes the actual progress that we have tested against what we have planned for. We should use measurable metrics, for instance numbers. 50 test cases done by this week.

Test Control: Taking action when test monitoring shows a problem, to make sure test objectives are still met.

Test monitoring and control:

Measuring and analyzing results

Monitoring and documenting test progress, test coverage achieved and the final criteria

Initiating corrective measures

Adjusting time and resource planning

Making decisions as to whether more tests are necessary

Test Monitoring and Control – Results:

Results typically include various types of test reports

Test progress report: A report that is created regularly every week, month, sprint. Shows how many tests done compared to planned, pass/fail, open defects by severity.

Test completion report: Created at milestone e.g. end of sprint, end of system test. It shows whether exit criteria are met yet, open defects by severity.

Test Analysis – Activities:

Test analysis focuses on determining what should be tested by analyzing the test basis and transforming it into well-defined, prioritized test conditions that are traceable to requirements and risks.

1. Review of the Test Basis: Look at the documentations like requirements to understand what the system should do.

2. Evaluation of Requirements and System Testability: Check if the requirement is clear and can be tested. Some requirements are not specific enough to be tested.

3. Risk Analysis: Decide what is risky and important to test first.

4. Identification of Features and Feature Sets: Identify what parts of the system need testing. So if we have a system that needs to be tested we should be able to figure out its individual components to be tested.

5. Detailing Test Objectives into Test Conditions: Turn “what to test” into specific test ideas.

6. Definition and Prioritization of Test Conditions: Decide which test conditions are most important. Which condition (input) is more important to be tested first.

7. Bi-Directional Traceability: Link requirements and test conditions both ways. Use some indentation in the documentation or something similar

Test Analysis – Results:

List of test conditions is defined and prioritized

(We know what to test and which tests are most important.)

Test conditions are linked to the test basis

(Each test can be traced back to the requirement, design, or other source.)

Test Design Activities:

- Specify test cases
- Prioritization of test cases, you can prioritize a set of test cases that are of the same theme for instance, the payment test cases should all have a high priority.
- Identify all the needed data. Normal valid data, edge cases, corrupt data, invalid.
- Decide where and how to run tests, including tools.
- Ensuring bi-directional traceability Link test cases back to requirements and vice versa.
- Test case should have a precondition to enter it, should have an expected result (output) post condition (the state of the system after the test case is finished).
- Linking Test Cases to Test Suites: Group related test cases together for easier execution.

Test Design Results:

- **(Sets of) test cases that cover the test conditions defined during test analysis:** for each test condition derived from the requirement during test analysis we have test case or test cases.
- **Bidirectional traceability to test conditions is important:** We should attach the test cases to the test condition and the opposite.
- **Hint: design abstract/logical test cases without concrete values for input data and expected results. Increases reusability!:** works as creating a template where all you have to change is the template.
- **Design/identification of necessary test data:** decide on normal data, edge, and invalid data.

- **Design of the test environment:** Where and how the test will be ran
- **Identification of infrastructure and tools :** you then know what is required to run the tests and test cases.
- **The extent to which these results are documented may vary**

Test Implementation – Activities:

1. Development and Prioritization of Test Scripts: prioritize the test cases and transform them into a test script.

2. Compilation/Creation of Test Suites from Test Procedures: Group related test scripts into test suites for easier execution.

3. Arranging the Test Suites in a Test Execution Schedule: Decide the order and timing to run test suites. Its based on the scenario timing not the severity of the test.

4. Setting Up the Test Environment: Prepare the servers, databases, network, and tools needed to run tests.

Examples: Test server with test database containing sample products

5. Verifying That Everything Required Is Set Up Correctly: Check that the environment, tools, and data are ready before testing.

Examples: Confirm database has sample users and products

6. Preparing Test Data and Loading It Into the Test Environment: Ensure all input data is available and correctly loaded for testing.

7. Verifying and Updating Bi-Directional Traceability: Check that test scripts still link correctly to the requirements/test conditions. Update links if needed.

8. Creation of Automated Test Scripts

Test Implementation – Results:

1. Test Scripts and Sequence of Those Scripts: The actual step-by-step test instructions are ready, and we know the order to run them.

2. Test Suite: A group of related test scripts that are executed together in one test run.

3. Test Execution Schedule: A plan showing the order and timing for running test suites during a test cycle.

Test Execution – Activities:

Simply: Run tests → Record results → Check bugs → Fix/retest → Update traceability
→ Check coverage

1. **Run the tests:** Execute the test suites following the test execution plan.
2. **Record information:** Note IDs, versions, tools, and resources used.
3. **Compare actual vs expected results**
4. **Identify defects and severity if something fails**
5. **Determine priority of fixes**
6. **Analyze causes of anomalies:** figure out why the test has failed, wrong input, wrong environment, bug on code.
7. **Report and record results:** whether it has passed or failed.
8. **Repeat tests if needed:** Re-run tests after fixes or for regression testing.
9. **Verify and update traceability:** Make sure links between requirements → test conditions → test cases → test procedures → test results are correct.
10. **Check coverage:** Did we test all requirements and code?, Identify unreachable code or gaps in testing.

Test Execution – Results:

1. Status Documentation: This has the status of each test script or test case (blocked, passed, failed, skipped, etc)
 2. Error Reports: Report any defects found during testing.
 3. Test Log: A record of what was tested, when, by whom, and results.
 4. Test Elements, Tools, and Equipment Used: Record which versions of software, tools, and environment were used.
 5. Evaluation Against End Criteria: Check if planned test strategy was executed and if end criteria are met. Decide if more tests are needed or criteria need adaptation.
 6. Practical Factors for Ending Testing: Often limited by time and cost. Make a decision: more testing or stop?
- Examples:

Test Completion – Activities:

Simply: Collect results & resources → Ensure defects documented → Archive and hand over → Analyze lessons → Improve process → Prepare final report

1. Collect Data from Completed Tests: Gather all results, logs, defects, and test resources from finished testing.
2. Check All Error Reports Completed: Make sure all defects found during testing are documented.
3. Create Change Requirements for Existing Errors: If errors need fixes in the product, create change requests. Meaning updating the requirements and beginning the whole testing process for this requirement again based on this error.
4. Finalize and Archive Test Environment and Assets: Save test data, test scripts, environments, and tools for future use.
5. Handover Test Resources: Give test assets, scripts, and knowledge to maintenance teams, other projects, or stakeholders. Share your findings.
6. Lessons Learned: Analyze what went well and what didn't. Identify improvements for future projects or product releases. Mention that the script was not written well, the requirements were not clear enough.
7. Improve the Test Process: Apply lessons learned to enhance the test approach.
8. Prepare Final Test Report: Summarize all test activities, results, defects, and lessons learned. Summary of test coverage and passed/failed tests. Overview of critical defects and how they were addressed

Test Completion – Results:

- **Collected test data & experience:** Gather all experience, results, test scripts, tools, and other information from testing. Like test logs and test cases that failed.
- **Final test report prepared:** Prepare a summary report of all testing activities, results, and defects.
- **Open issues documented for next projects**
- **Test resources finalized and archived:** archive your findings and improvements for future use.